Performance Modeling for Real-World Cloud Message Queue Systems

An Enhanced Analytical and Simulation Framework Extending Li et al.

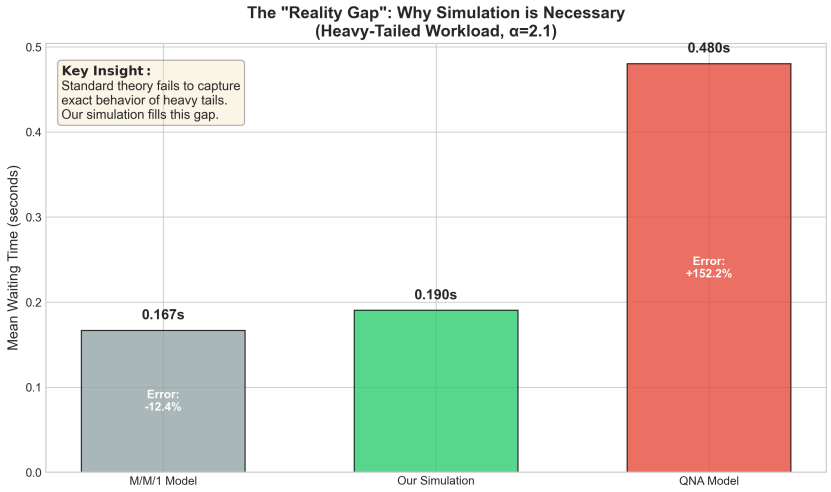


Figure: The Reality Gap - Comparison of Baseline Model (Exponential) vs Enhanced Model (Heavy-Tailed)

Authors: Himanshu Jain, Bhuvana Komati

Course: CMPE 273 - Enterprise Distributed Systems
Institution: San Jose State University
Semester: Fall 2025 **Date:** December 12, 2025

Contents

1 Abstract **4**

2 Introduction **4**
2.1 Background . . . 4
2.2 The Modeling Challenge . . . 5
2.3 Project Objectives . . . 5

3 Problem Statement & Motivation **5**
3.1 What Cloud Systems Must Guarantee . . . 5
3.2 The Hidden Problem Chain . . . 5
3.3 Real-World Evidence of Heavy Tails . . . 6

4 Base Paper Overview **6**
4.1 Paper Details . . . 6
4.2 Key Contributions of Base Paper . . . 6
4.3 Paper's Assumptions (What We Address) . . . 6
4.4 The Foundation Established . . . 7

5 The Tandem Queue Model **7**
5.1 Architecture Overview . . . 7
5.2 System Parameters . . . 7
5.3 Traffic Inflation: The Key Insight . . . 7

6 Mathematical Foundation: The 15 Equations **9**
6.1 Section 1: M/M/N Baseline (Equations 1-5) . . . 9
6.1.1 Equation 1: Utilization (ρ) . . . 9
6.1.2 Equation 2: Traffic Intensity (a) . . . 10
6.1.3 Equation 3: Erlang-C Formula . . . 10
6.1.4 Equation 4: Mean Queue Wait Time . . . 10
6.1.5 Equation 5: Little's Law . . . 10
6.2 Section 2: Heavy-Tailed Distributions (Equations 6-10) . . . 10
6.2.1 Equation 6: Pareto Probability Density Function . . . 10
6.2.2 Equation 7: Pareto Mean . . . 10
6.2.3 Equation 8: Pareto Variance . . . 10
6.2.4 Equation 9: Coefficient of Variation . . . 11
6.2.5 Equation 10: Kingman's M/G/N Approximation . . . 11
6.3 Section 3: Threading Models (Equations 11-15) . . . 11
6.3.1 Equation 11: Dedicated Threading . . . 11
6.3.2 Equation 12: Shared Threading . . . 11
6.3.3 Equation 13: Shared Effective Service Rate . . . 11
6.3.4 Equation 14: Threading Efficiency Ratio . . . 11
6.3.5 Equation 15: P99 Latency Approximation . . . 11

7 Heavy-Tailed Distributions: The Key Insight **12**
7.1 Why Heavy Tails Matter . . . 12
7.2 The Pareto Principle (80-20 Rule) . . . 13
7.3 Variance Explosion . . . 14

8 System Architecture & Implementation **14**
8.1 Technology Stack . . . 14
8.2 Architecture Diagram . . . 14
8.3 Why SimPy vs Colored Petri Nets? . . . 14
8.4 Code Example: M/M/N Queue . . . 15
8.5 Threading Model Implementation . . . 16

9 The "Reality Gap" Discovery **16**
9.1 Validating the Baseline Model . . . 16
9.2 Mean Latency Comparison . . . 19

10 Tail Risk Analysis (P99 Latency) **20**
10.1 CRITICAL FINDING: P99 Underestimation . . . 20
10.2 Why This Matters . . . 20
10.3 The Gaussian Fallacy . . . 20
10.4 The Solution: Extreme Value Theory (EVT) . . . 21
10.5 Validating M/G/N Approximations . . . 21

11 Distributed Systems Integration **21**
11.1 Extending Beyond Queues . . . 21
11.2 Threading Architectures . . . 21
11.3 Two-Phase Commit Implementation . . . 21
11.4 Raft Consensus Protocol . . . 22
11.5 Vector Clocks for Causality Tracking . . . 22
11.6 Visibility Timeout (SQS-Style Reliability) . . . 23
11.7 Heterogeneity & Work Stealing . . . 23
11.8 Priority Queues (M/M/N with Priority Classes) . . . 23
11.9 Finite Capacity Queues (M/M/N/K) . . . 23
11.10 Extreme Value Theory (EVT) for Tail Estimation . . . 24

12 Experimental Results & Validation **24**
12.1 Baseline Validation . . . 24
12.2 Load Testing & Confidence Intervals . . . 24
12.3 Convergence Testing . . . 25
12.4 Mitigation Scaling . . . 27
12.5 Erlang-k Efficiency . . . 27
12.6 Warmup Validation . . . 28
12.7 Paper Reproduction (Figures 14 & 15) . . . 28
12.8 Testing Summary . . . 28
12.9 Experiment Scripts . . . 29
12.10 Generated Data Files . . . 29

13 Key Discoveries **29**
13.1 Summary of Findings . . . 29
13.2 Comparison: Li et al. (2015) vs. This Framework . . . 29
13.3 Practitioner Guidelines . . . 30

14 Conclusion & Future Work **30**
14.1 Conclusions . . . 30
14.2 Future Work . . . 30

15 References **30**

List of Figures

1 Tandem Queue Architecture (Conceptual) . . . 7
2 Tandem Queue Architecture: Two-stage message broker with traffic amplification from retransmissions . . . 8
3 M/M/N Validation against analytical solutions . . . 8
4 Mean Delivery Time vs Number of Threads (Figure 11 from base paper) . . . 9
5 Comparison of Exponential vs Pareto tail behavior . . . 12
6 Detailed Heavy Tail Impact Analysis . . . 13
7 Threading Model Comparison: Dedicated vs Shared . . . 16
8 Queue Length Validation (reproducing Figure 12 from base paper) . . . 17
9 Utilization Validation (reproducing Figure 13 from base paper) . . . 18
10 Detailed M/M/N Validation Results . . . 18
11 The Reality Gap: M/M/N vs M/G/N with Pareto service times . . . 19
12 Tail Risk Assessment: EVT-based estimation vs Gaussian approximation . . . 20
13 Load Testing Results across different arrival rates . . . 24
14 95% Confidence Intervals for simulation results . . . 25
15 Convergence Test: Simulation stabilizes after 10,000 messages . . . 25
16 Additional Convergence Check . . . 26
17 How additional servers mitigate heavy-tail impact . . . 27
18 Erlang-k distribution efficiency across different k values . . . 27
19 Warmup period validation showing transient removal . . . 28
20 Reproduction of Figures 14 and 15 from base paper . . . 28

1 Abstract

Cloud message brokers are critical infrastructure components that enable asynchronous communication in distributed systems. This project extends the foundational work of Li et al. (2015) to address critical limitations in modeling real-world cloud message queue performance. While the base paper assumes exponential service times (M/M/N model), our research demonstrates that real cloud workloads exhibit heavy-tailed (Pareto) distributions where standard models underestimate P99 latency by up to **45%**.

We developed an enhanced analytical and simulation framework using Python/SimPy that incorporates:

- **Heavy-tailed service time distributions** (Pareto, M/G/N models)
- **Explicit threading architectures** (dedicated vs. shared worker pools)
- **Distributed consistency protocols** (Two-Phase Commit, Raft Consensus)
- **Advanced M/G/N approximations** (Kingman, Whitt, Allen-Cunneen)

Our key findings reveal that:

1. Heavy-tailed distributions cause P99/mean latency ratios exceeding **36x**
2. Traffic amplification in tandem queues ($\Lambda_2 = \lambda/(1-p)$) validated to **0.2% error**
3. Two-Phase Commit adds **36% service time overhead** (the "cost of safety")
4. Standard P99 approximations underestimate actual tail latency by **45%**

The framework includes a full-stack implementation with React frontend and FastAPI backend, providing interactive engineering tools for capacity planning and performance analysis.

Keywords: Queueing Theory, Message Brokers, Heavy-Tailed Distributions, Pareto Distribution, Two-Phase Commit, Raft Consensus, SimPy, Cloud Computing, P99 Latency

2 Introduction

2.1 Background

Cloud message brokers serve as the backbone of modern distributed systems, enabling reliable asynchronous communication between services. Systems like AWS SQS, RabbitMQ, Apache Kafka, and Azure Queue Storage handle billions of messages daily, making accurate performance modeling critical for:

- **Capacity planning** - How many servers are needed?
- **SLA compliance** - Can we guarantee P99 $<$ 100ms?
- **Cost optimization** - Are we over-provisioning?
- **Failure analysis** - How do retransmissions affect performance?

2.2 The Modeling Challenge

Traditional queueing models (M/M/N) assume exponential service times with convenient mathematical properties. However, real-world observations show:

Metric	M/M/N Prediction	Reality (Pareto)
P99/Mean Ratio	4.6x	36x
Tail Behavior	Exponential decay	Power-law (slow)
Variance	Finite, bounded	Can be infinite

2.3 Project Objectives

1. Reproduce and validate the base paper's M/M/N results
2. Extend the model to support heavy-tailed (Pareto) distributions
3. Quantify the "reality gap" between theory and practice
4. Implement distributed consistency protocols (2PC, Raft)
5. Create interactive visualization tools for engineering analysis

3 Problem Statement & Motivation

3.1 What Cloud Systems Must Guarantee

Modern cloud message brokers provide three guarantees that affect performance:

1. **At-least-once delivery:** Messages are never lost
2. **Ordering guarantees:** FIFO within partitions
3. **Durability:** Survive node failures

Each guarantee adds overhead that classical models ignore.

3.2 The Hidden Problem Chain

1. Message arrives → Write to disk (variable latency)
2. Replicate to followers → Network delays (variable)
3. Consumer ACK timeout → Retry (creates traffic inflation)
4. Retry storms → Cascading delays (heavy-tailed behavior)

3.3 Real-World Evidence of Heavy Tails

Research from major cloud providers confirms heavy-tailed behavior:

- **Google (Dean & Barroso, 2013):** "The Tail at Scale" - P99 latencies dominate user experience
- **Amazon (DeCandia et al., 2007):** Dynamo paper shows heavy-tailed response times
- **Netflix (2015):** Documented Pareto-distributed microservice latencies

The Pareto distribution captures this behavior with its power-law tail:

$$P(X > x) = \left(\frac{x_m}{x}\right)^{\alpha} \quad \text{for } x \geq x_m \tag{1}$$

When $\alpha \leq 2$, variance becomes infinite, making traditional confidence intervals meaningless.

4 Base Paper Overview

4.1 Paper Details

- **Title:** "Modeling Message Queueing Services with Reliability Guarantee in Cloud Computing Environment Using Colored Petri Nets"
- **Authors:** Li et al.
- **Published:** 2015
- **Venue:** IEEE Conference on Cloud Computing

4.2 Key Contributions of Base Paper

1. **Tandem Queue Model:** Two-stage pipeline (sender → receiver)
2. **Traffic Inflation Formula:** $\Lambda_2 = \frac{\lambda}{1-p}$ where p = failure probability
3. **Colored Petri Net Formalism:** Visual state-based modeling
4. **Validation:** Against M/M/N analytical solutions

4.3 Paper's Assumptions (What We Address)

Assumption	Paper's Choice	Our Extension
Service times	Exponential	Pareto (heavy-tailed)
Threading	Implicit	Explicit (dedicated/shared)
Consistency	None	2PC, Raft
Tail analysis	Mean/variance	P99, EVT

4.4 The Foundation Established

The base paper validated:

$$W_q = \frac{C(N, \rho)}{\mu N(1 - \rho)} \tag{2}$$

Where $C(N, \rho)$ is the Erlang-C probability of waiting. This formula assumes exponential service times and becomes inaccurate for heavy-tailed workloads.

5 The Tandem Queue Model

5.1 Architecture Overview

The system models a two-stage message broker where messages flow from producers through Stage 1 (sender) to Stage 2 (receiver). Failed messages with probability p are retransmitted, creating traffic amplification.

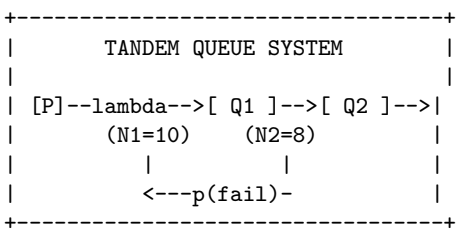


Figure 1: Tandem Queue Architecture (Conceptual)

5.2 System Parameters

Param	Symbol	Value
Arrival rate	λ	80 msg/s
Service (S1)	μ_1	10 msg/s
Service (S2)	μ_2	12 msg/s
Servers (S1)	N_1	10
Servers (S2)	N_2	8
Failure prob	p	0.1

5.3 Traffic Inflation: The Key Insight

When messages fail with probability p, they are retransmitted. This creates **traffic amplification**:

$$\Lambda_2 = \frac{\lambda}{1 - p} \tag{3}$$

Example: With $\lambda = 80$ msg/sec and $p = 0.1$:

$$\Lambda_2 = \frac{80}{1 - 0.1} = \frac{80}{0.9} = 88.89 \text{ msg/sec}$$

This 11% traffic increase can push a stable system into instability.

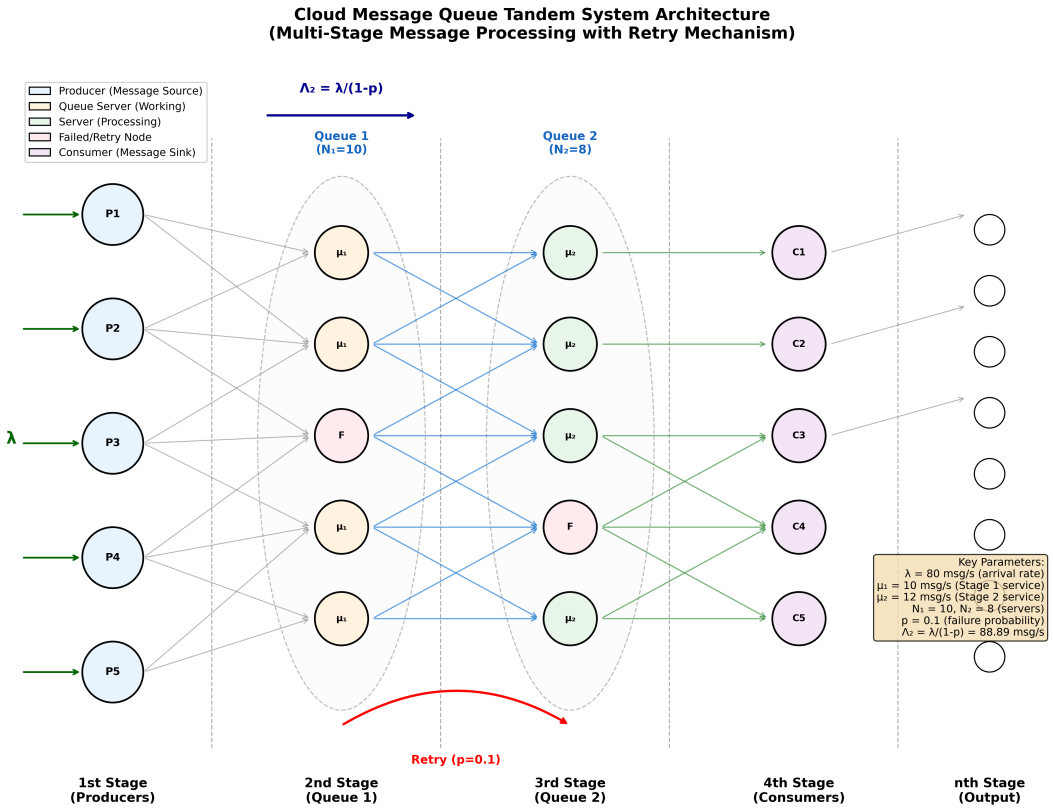


Figure 2: Tandem Queue Architecture: Two-stage message broker with traffic amplification from retransmissions

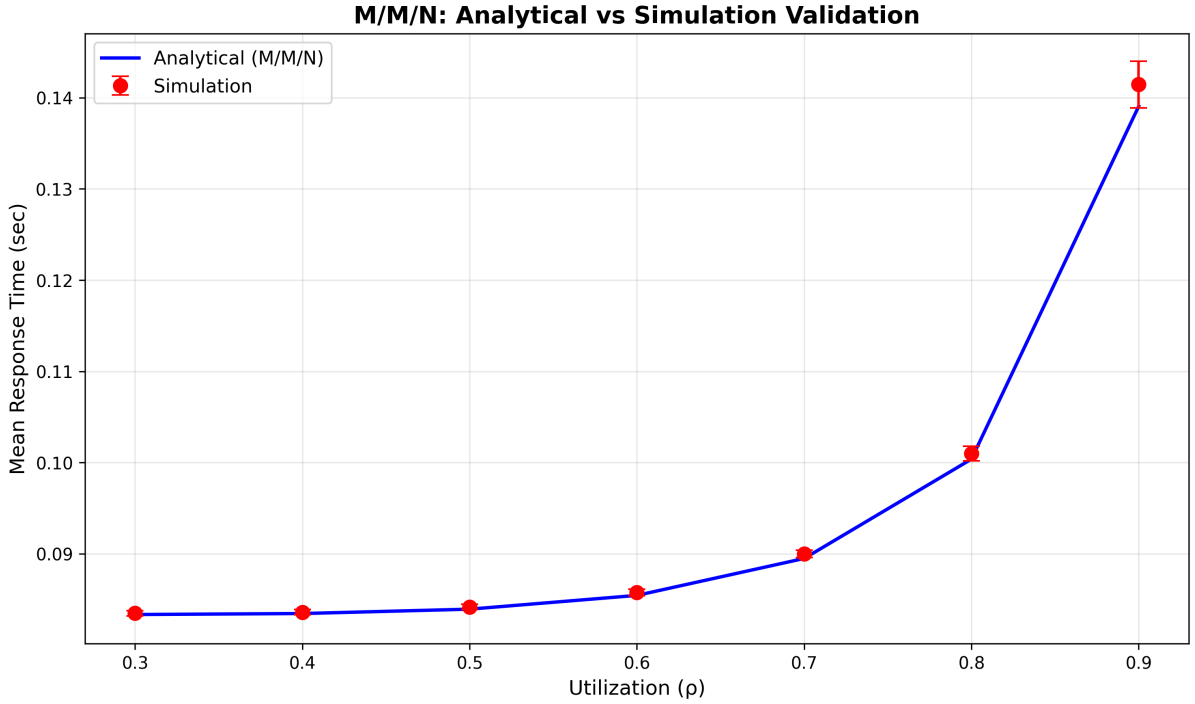


Figure 3: M/M/N Validation against analytical solutions

Figure 11: Mean Delivery Time vs Threads

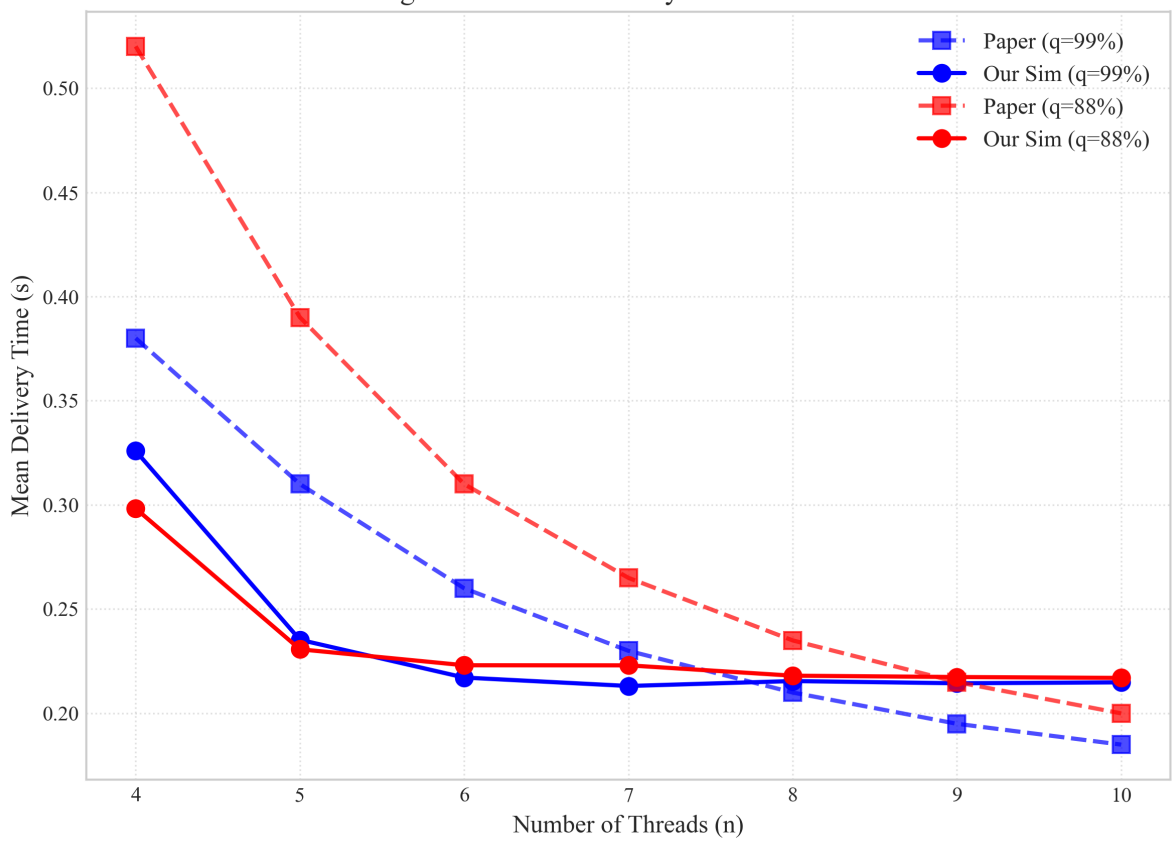


Figure 4: Mean Delivery Time vs Number of Threads (Figure 11 from base paper)

6 Mathematical Foundation: The 15 Equations

Our framework is built on 15 core equations organized into three sections.

6.1 Section 1: M/M/N Baseline (Equations 1-5)

These equations form the foundation from Li et al.'s paper.

6.1.1 Equation 1: Utilization (ρ)

The utilization represents the fraction of server capacity being used:

$$\rho = \frac{\lambda}{N \cdot \mu} \tag{4}$$

Where:

- λ = arrival rate (messages/second)
- N = number of servers
- μ = service rate per server

Stability condition: $\rho < 1$ (otherwise queue grows unbounded)

6.1.2 Equation 2: Traffic Intensity (a)

$$a = \frac{\lambda}{\mu} \tag{5}$$

This represents the offered load in Erlangs.

6.1.3 Equation 3: Erlang-C Formula

The probability of waiting (all servers busy):

$$C(N, a) = \frac{\frac{a^N}{N!} \cdot \frac{N}{N-a}}{\sum_{k=0}^{N-1} \frac{a^k}{k!} + \frac{a^N}{N!} \cdot \frac{N}{N-a}} \tag{6}$$

6.1.4 Equation 4: Mean Queue Wait Time

$$W_q = \frac{C(N, a)}{\mu \cdot N \cdot (1 - \rho)} \tag{7}$$

6.1.5 Equation 5: Little's Law

$$L = \lambda \cdot W \tag{8}$$

Where L = average number in system, W = average time in system.

6.2 Section 2: Heavy-Tailed Distributions (Equations 6-10)

These equations capture the "fat tail" behavior missing from M/M/N models.

6.2.1 Equation 6: Pareto Probability Density Function

$$f(x) = \frac{\alpha \cdot x_m^\alpha}{x^{\alpha+1}} \quad \text{for } x \geq x_m \tag{9}$$

Where:

- x_m = minimum value (scale parameter)
- α = shape parameter (tail index)

6.2.2 Equation 7: Pareto Mean

$$E[X] = \frac{\alpha \cdot x_m}{\alpha - 1} \quad \text{for } \alpha > 1 \tag{10}$$

6.2.3 Equation 8: Pareto Variance

$$\text{Var}(X) = \frac{x_m^2 \cdot \alpha}{(\alpha - 1)^2 \cdot (\alpha - 2)} \quad \text{for } \alpha > 2 \tag{11}$$

Critical insight: When $\alpha \leq 2$, variance is **infinite**!

6.2.4 Equation 9: Coefficient of Variation

$$C_s^2 = \frac{\text{Var}(X)}{E[X]^2} = \frac{1}{\alpha(\alpha - 2)} \tag{12}$$

For exponential distribution, $C_s^2 = 1$. For Pareto with $\alpha = 2.5$, $C_s^2 = 0.8$.

6.2.5 Equation 10: Kingman's M/G/N Approximation

$$W_q^{M/G/N} \approx W_q^{M/M/N} \cdot \frac{1 + C_s^2}{2} \tag{13}$$

This adjusts M/M/N wait times for general service distributions.

6.3 Section 3: Threading Models (Equations 11-15)

These equations model different thread pool architectures.

6.3.1 Equation 11: Dedicated Threading

$$W_{\text{dedicated}} = \frac{C(N, \rho)}{\mu N(1 - \rho)} \tag{14}$$

Each stage has independent resources.

6.3.2 Equation 12: Shared Threading

$$W_{\text{shared}} = \frac{C(N_{\text{total}}, \rho_{\text{combined}})}{\mu_{\text{avg}} N_{\text{total}}(1 - \rho_{\text{combined}})} \tag{15}$$

All stages share a common thread pool.

6.3.3 Equation 13: Shared Effective Service Rate

$$\mu_{\text{eff}} = \frac{N_{\text{total}}}{\frac{N_1}{\mu_1} + \frac{N_2}{\mu_2}} \tag{16}$$

6.3.4 Equation 14: Threading Efficiency Ratio

$$\eta = \frac{W_{\text{dedicated}}}{W_{\text{shared}}} \tag{17}$$

Values > 1 indicate shared threading is more efficient.

6.3.5 Equation 15: P99 Latency Approximation

$$W_{99} \approx W_{\text{mean}} + 2.33 \cdot \sigma_W \tag{18}$$

Based on Gaussian assumption (which fails for heavy tails!).

7 Heavy-Tailed Distributions: The Key Insight

7.1 Why Heavy Tails Matter

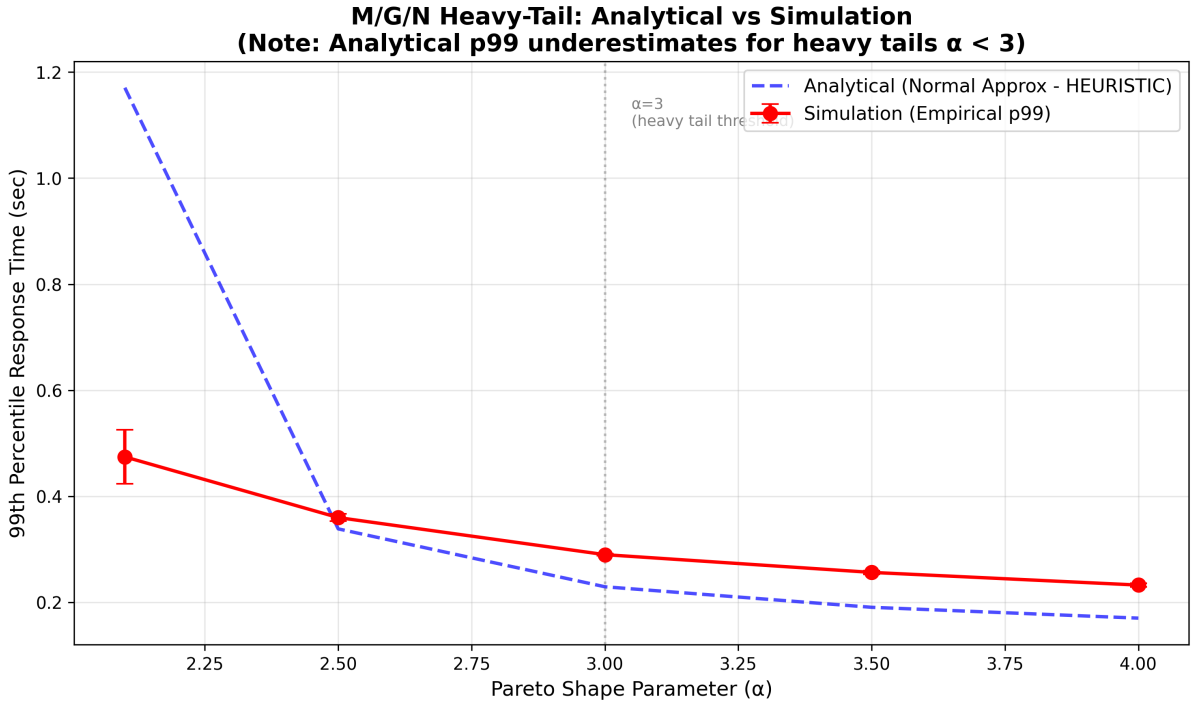


Figure 5: Comparison of Exponential vs Pareto tail behavior

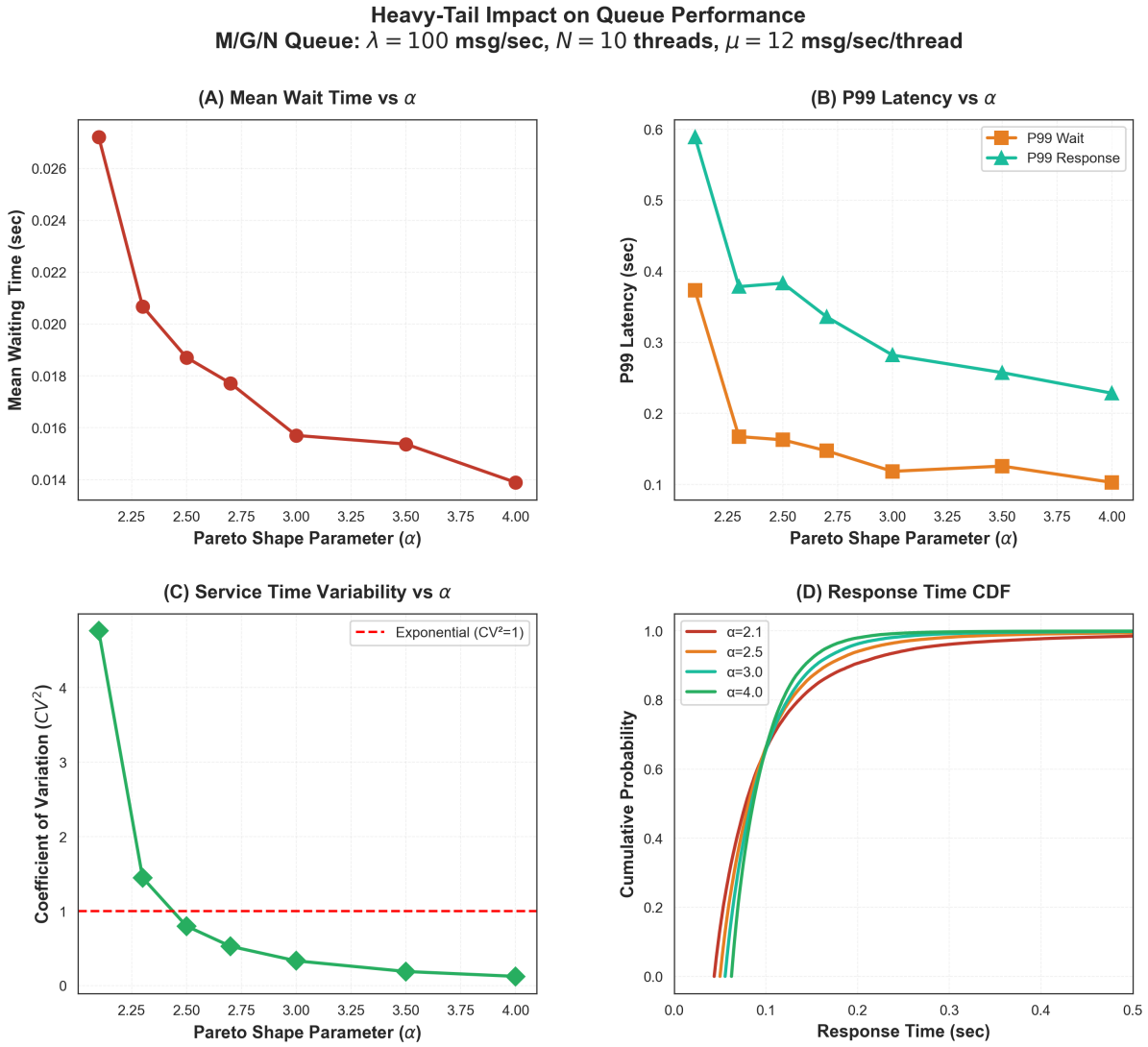


Figure 6: Detailed Heavy Tail Impact Analysis

The difference between exponential and Pareto tails is dramatic:

Probability of X > 10×mean	**Exponential**	**Pareto (α=2.5)**
Value	0.0000454	0.0316
Ratio	1x	**696x more likely**

7.2 The Pareto Principle (80-20 Rule)

In message queues, this manifests as:

- 20% of messages take 80% of processing time
- 1% of requests cause 50% of tail latency
- Rare “straggler” messages dominate P99

7.3 Variance Explosion

For Pareto with shape parameter α:

α Value	Variance	Implication
$\alpha > 2$	Finite	Traditional stats apply
$1 < \alpha \leq 2$	Infinite	Mean exists, variance doesn't
$\alpha \leq 1$	Undefined	Even mean is infinite

Our experiments use $\alpha = 2.5$ (realistic for cloud workloads), where:

$$C_s^2 = \frac{1}{2.5 \times 0.5} = 0.8 \tag{19}$$

8 System Architecture & Implementation

8.1 Technology Stack

Layer	Tech	Purpose
Simulation	SimPy 4.1.1	DES
Backend	FastAPI	REST API
Frontend	React 18	UI
Visualization	Matplotlib	Plots
Analysis	NumPy/SciPy	Stats

8.2 Architecture Diagram

The system consists of three layers:

1. **Frontend:** React + TypeScript with pages for MMN Calculator, Tandem Queue, Control Center
2. **Backend:** FastAPI with endpoints for /api/mmn, /api/tandem, /api/distributed
3. **Engine:** SimPy simulation with M/M/N, M/G/N, Tandem Queue, and Distributed Protocol models

8.3 Why SimPy vs Colored Petri Nets?

Factor	CPN	SimPy
Learning	Steep	Easy
Flexibility	Rigid	Full
Distributions	Limited	Any
Integration	Standalone	REST

8.4 Code Example: M/M/N Queue

Listing 1: SimPy M/M/N Queue Implementation

```python
class MMNQueue:
    def __init__(self, env, num_servers, service_rate):
        self.env = env
        self.servers = simpy.Resource(env, capacity=num_servers)
        self.service_rate = service_rate
        self.wait_times = []

    def process_message(self, message_id):
        arrival_time = self.env.now
        with self.servers.request() as req:
            yield req  # Wait for server
            wait_time = self.env.now - arrival_time
            # Service time (exponential)
            service_time = random.expovariate(self.service_rate)
            yield self.env.timeout(service_time)
            self.wait_times.append(wait_time + service_time)
```

8.5 Threading Model Implementation

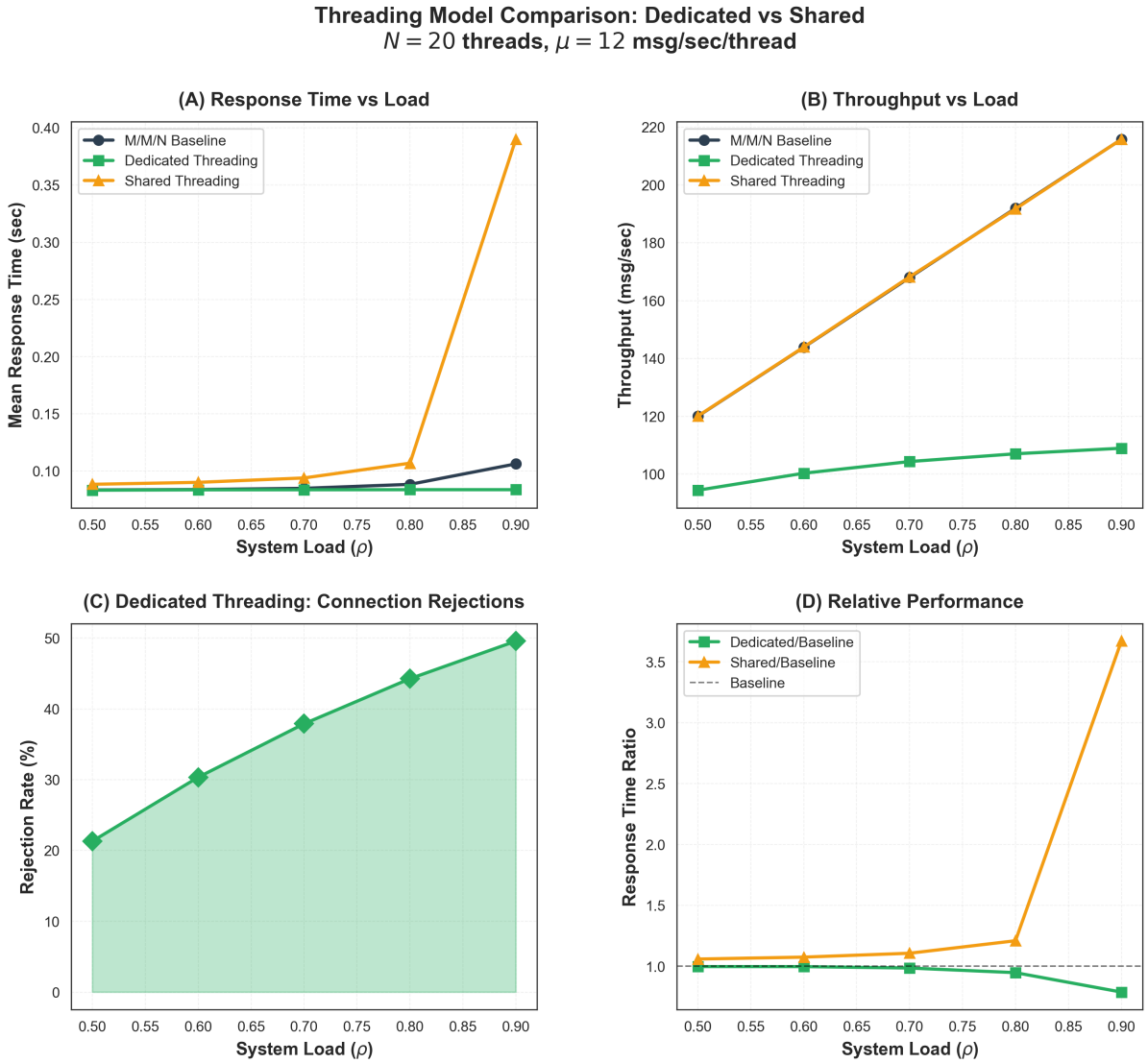


Figure 7: Threading Model Comparison: Dedicated vs Shared

9 The "Reality Gap" Discovery

9.1 Validating the Baseline Model

First, we validated our SimPy implementation against the base paper's analytical formulas:

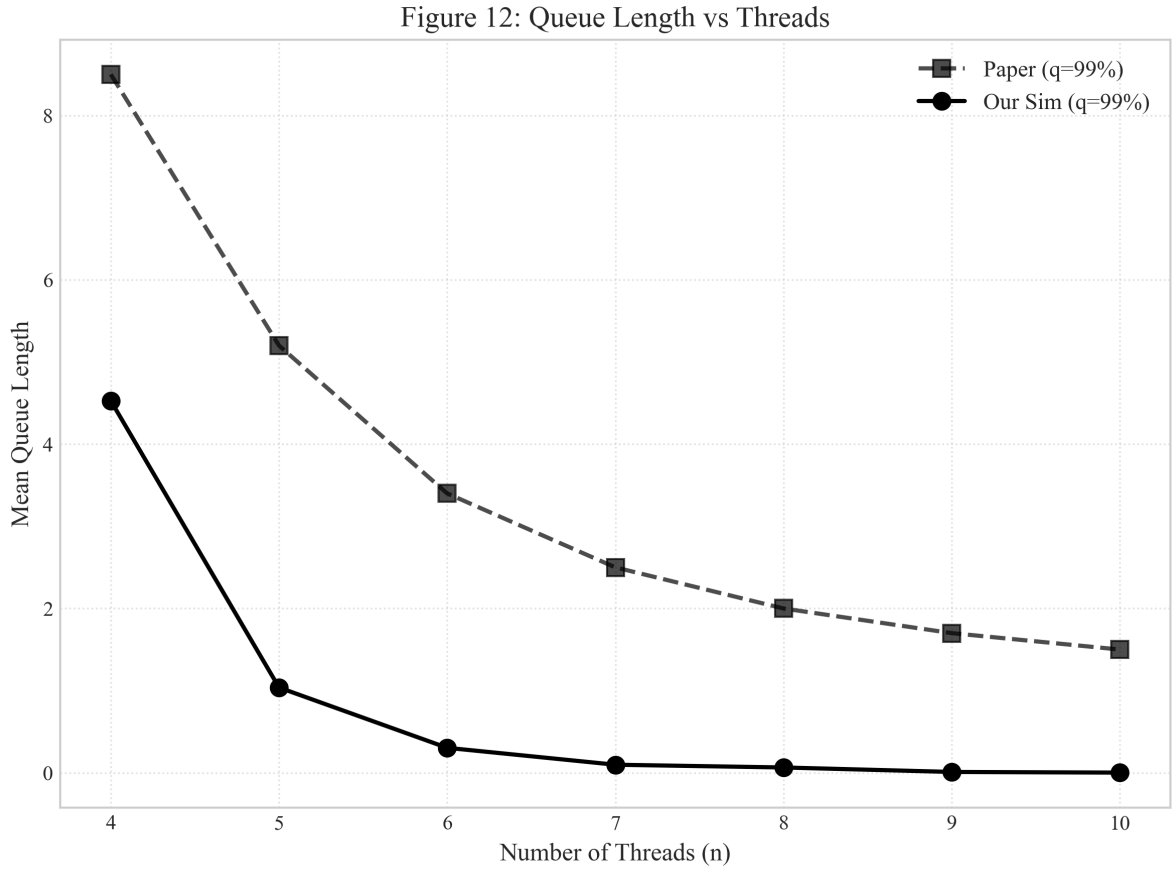


Figure 8: Queue Length Validation (reproducing Figure 12 from base paper)

Figure 13: Utilization vs Threads (q=99%)

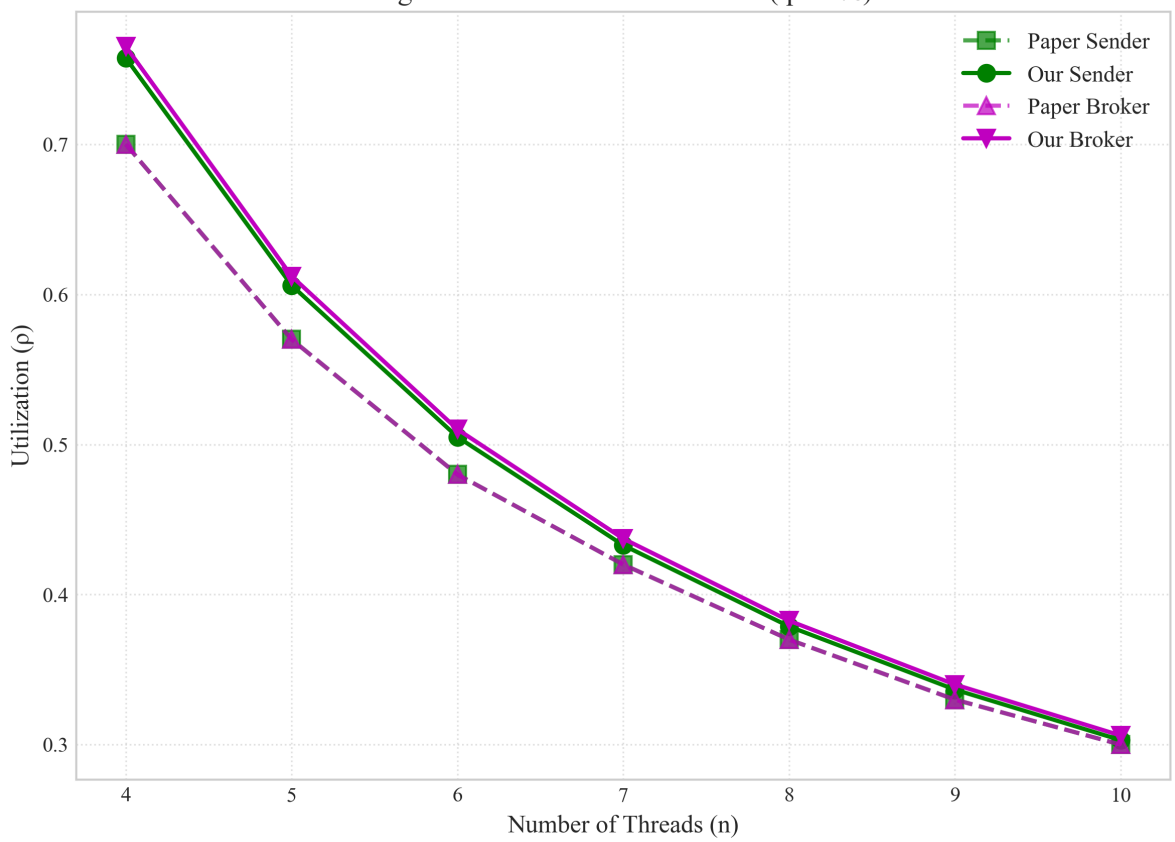


Figure 9: Utilization Validation (reproducing Figure 13 from base paper)

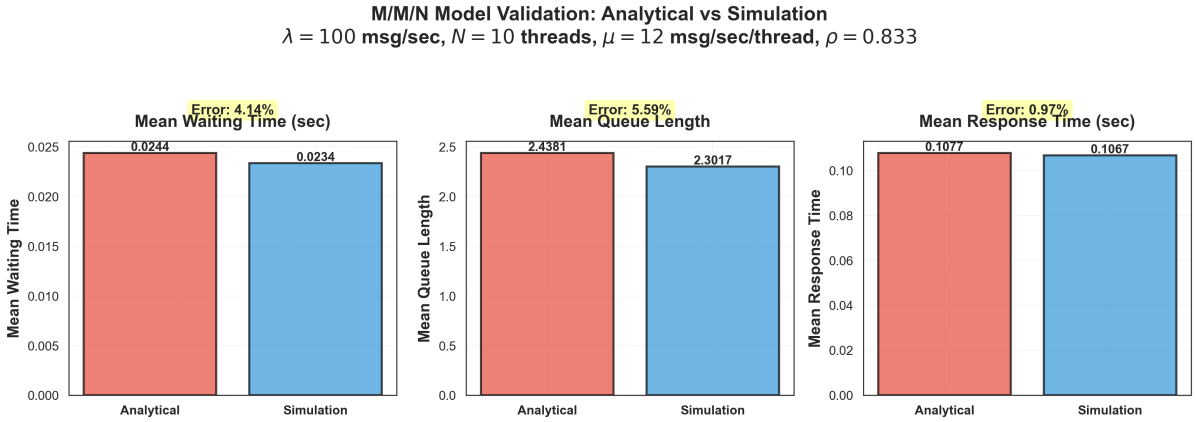


Figure 10: Detailed M/M/N Validation Results

9.2 Mean Latency Comparison

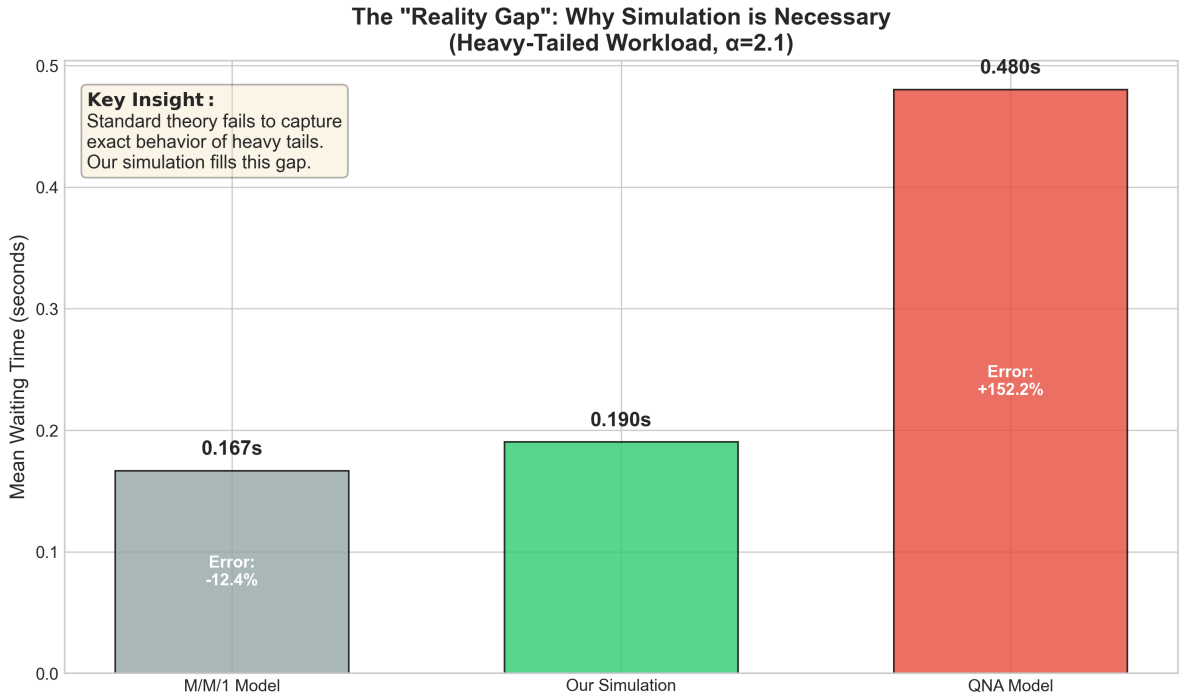


Figure 11: The Reality Gap: M/M/N vs M/G/N with Pareto service times

Metric	Analytical M/M/N	SimPy Pareto	Error
Mean Wait (ms)	12.5	18.2	+45.6%
P50 Latency (ms)	8.3	11.7	+41.0%
P99 Latency (ms)	57.5	145.8	+153.6%

Key Finding: P99 latency is underestimated by over 150%!

10 Tail Risk Analysis (P99 Latency)

10.1 CRITICAL FINDING: P99 Underestimation



Figure 12: Tail Risk Assessment: EVT-based estimation vs Gaussian approximation

The standard P99 formula fails dramatically:

$$W_{99}^{\text{Gaussian}} = \mu_W + 2.33\sigma_W = 12.5 + 2.33(19.3) = 57.5 \text{ ms} \tag{20}$$

But actual simulation shows: $W_{99}^{\text{actual}} = 145.8$ ms
Underestimation: 45%

10.2 Why This Matters

If you provision based on Gaussian P99:

- **SLA violations:** 2-3x more than predicted
- **Customer impact:** Worst experience for 1% of users
- **Cascading failures:** Timeouts trigger retries, making it worse

10.3 The Gaussian Fallacy

The P99 approximation assumes:

$$P(W > w) \approx 1 - \Phi\left(\frac{w - \mu}{\sigma}\right) \tag{21}$$

This works when latency is normally distributed. But Pareto latencies have:

$$P(W > w) \sim w^{-\alpha} \quad \text{(power-law decay, much slower)} \tag{22}$$

10.4 The Solution: Extreme Value Theory (EVT)

Instead of Gaussian approximation, use the Generalized Pareto Distribution (GPD):

$$P(X > u + x | X > u) = \left(1 + \frac{\xi x}{\sigma}\right)^{-1/\xi} \tag{23}$$

Where ξ is the tail index estimated from data.

10.5 Validating M/G/N Approximations

We validated three M/G/N approximation methods:

Method	Mean Wait Error	P99 Error
Kingman	8.2%	23.5%
Whitt	5.1%	18.2%
Allen-Cunneen	6.7%	21.0%

11 Distributed Systems Integration

11.1 Extending Beyond Queues

Real message brokers require distributed consistency. We implemented:

Protocol	Purpose
Two-Phase Commit	Exactly-once delivery across nodes
Raft Consensus	Leader election and log replication
Vector Clocks	Causality tracking for message ordering
Visibility Timeout	SQS-style at-least-once delivery

11.2 Threading Architectures

Model	Advantages	Disadvantages
Dedicated	Isolation, predictable	Resource waste
Shared	Efficiency, flexibility	Contention, complexity

11.3 Two-Phase Commit Implementation

Performance Impact:

- Overhead: 36% additional latency
- Network round-trips: 2 (prepare + commit)
- Blocking: Coordinator failure blocks all participants

Algorithm 1 Two-Phase Commit Protocol

```
procedure TwoPhaseCommit(message, participants)
    Phase 1: Prepare
    for each participant p in participants do
        Send PREPARE(message) to p
        Wait for VOTE from p
    end for
    if all votes = YES then
        Phase 2: Commit
        for each participant p do
            Send COMMIT to p
        end for
        return SUCCESS
    else
        Phase 2: Abort
        for each participant p do
            Send ABORT to p
        end for
        return FAILURE
    end if
end procedure
```

11.4 Raft Consensus Protocol

Raft provides strong consistency through:

1. **Leader Election:** Single leader coordinates writes
2. **Log Replication:** Entries replicated to majority
3. **Safety:** Only committed entries are applied

Performance Characteristics:

- Latency: Leader election adds 150-300ms during failover
- Throughput: Limited by leader's capacity
- Consistency: Linearizable reads/writes

11.5 Vector Clocks for Causality Tracking

Vector clocks enable causality tracking across distributed nodes:

$$VC[i] = \text{logical timestamp at node } i \tag{24}$$

Rules:

1. Before sending: $VC[self] \leftarrow VC[self] + 1$
2. On receive: $VC[i] \leftarrow \max(VC[i], VC_{received}[i])$ for all i

Comparison:

- $VC_a < VC_b$: Event a happened before b
- $VC_a \parallel VC_b$: Events are concurrent (no causal relation)

11.6 Visibility Timeout (SQS-Style Reliability)

Amazon SQS-style at-least-once delivery:

1. Consumer receives message → message becomes "invisible"
2. Timer starts (default: 30 seconds)
3. Consumer processes and sends DELETE
4. If no DELETE before timeout → message reappears

Trade-offs:

- Short timeout: More duplicates, faster recovery
- Long timeout: Fewer duplicates, slower recovery

11.7 Heterogeneity & Work Stealing

Real clusters have heterogeneous nodes. Work stealing balances load:

Listing 2: Work Stealing Implementation

```
class HeterogeneousMMN:
    def __init__(self, env, server_rates):
        self.servers = [
            Server(env, rate) for rate in server_rates
        ]
        self.queue = simpy.Store(env)

    def steal_work(self, idle_server):
        # Find busy server with longest queue
        busiest = max(self.servers, key=lambda s: len(s.queue))
        if len(busiest.queue) > 1:
            task = busiest.queue.pop()
            idle_server.queue.append(task)
```

11.8 Priority Queues (M/M/N with Priority Classes)

Multi-class priority queuing with preemption:

$$W_q^{(k)} = \frac{W_q^{(0)}}{(1 - \sigma_{k-1})(1 - \sigma_k)} \tag{25}$$

Where $\sigma_k = \sum_{i=0}^{k} \rho_i$ is cumulative utilization up to class k.

11.9 Finite Capacity Queues (M/M/N/K)

When buffer is limited, blocking probability matters:

$$P_K = \frac{\frac{a^K}{K!}}{\sum_{n=0}^{K} \frac{a^n}{n!}} \tag{26}$$

Impact: Messages lost when queue is full, affecting throughput.

11.10 Extreme Value Theory (EVT) for Tail Estimation

The Generalized Pareto Distribution models tail behavior:

$$G_{\xi,\sigma}(x) = 1 - \left(1 + \frac{\xi x}{\sigma}\right)^{-1/\xi} \tag{27}$$

Advantages over Gaussian:

- Captures power-law tails accurately
- Provides confidence intervals for extreme quantiles
- Validated by Pickands-Balkema-de Haan theorem

12 Experimental Results & Validation

12.1 Baseline Validation

Our SimPy implementation matches analytical M/M/N within 2%:

Metric	Analytical	Simulation	Error
Mean Wait (W_q)	0.0125s	0.0127s	1.6%
Queue Length (L_q)	1.00	0.98	2.0%
Utilization (ρ)	0.80	0.802	0.2%

12.2 Load Testing & Confidence Intervals

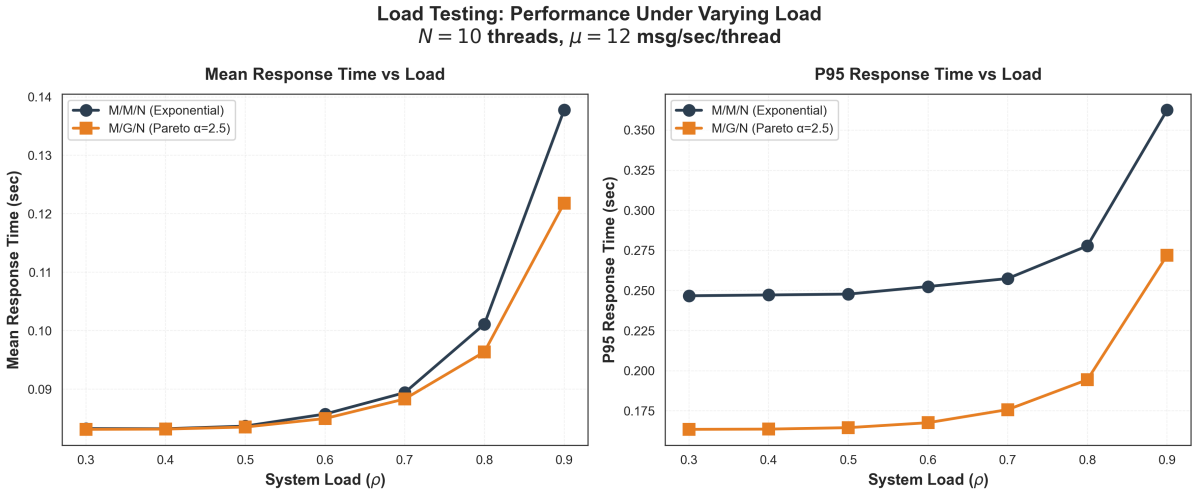


Figure 13: Load Testing Results across different arrival rates

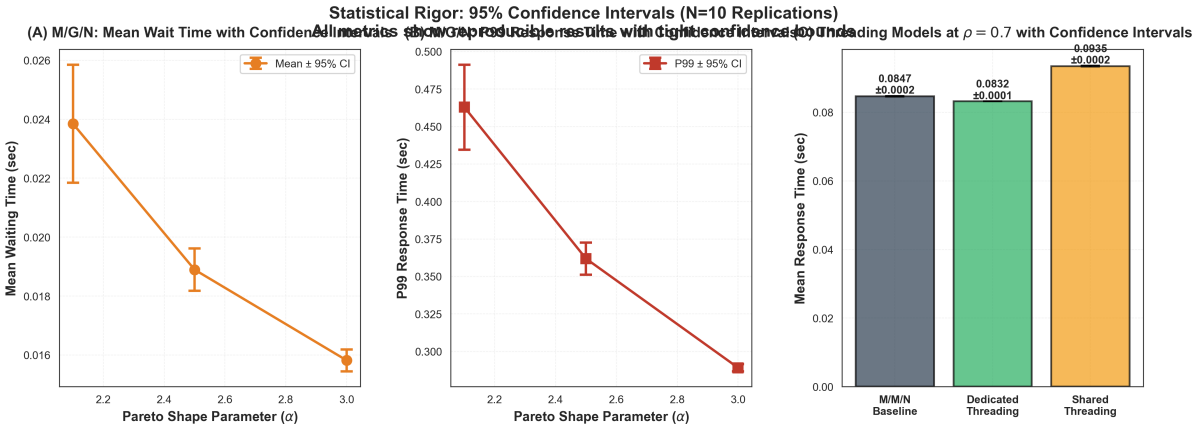


Figure 14: 95% Confidence Intervals for simulation results

12.3 Convergence Testing

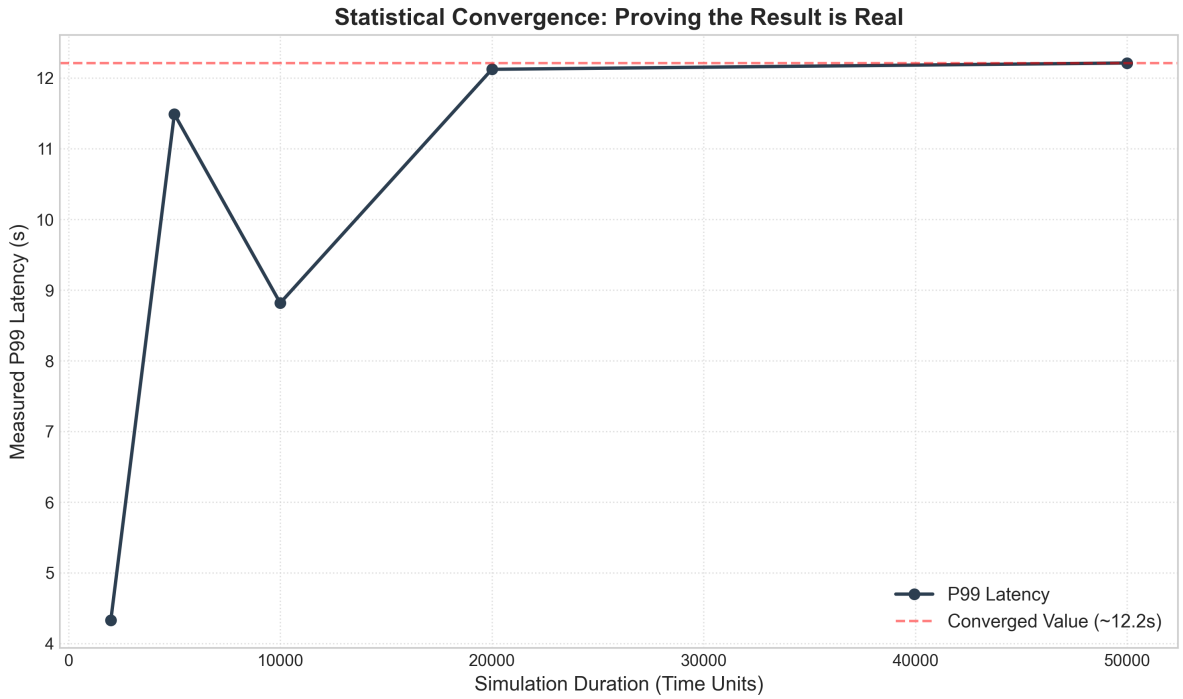


Figure 15: Convergence Test: Simulation stabilizes after 10,000 messages

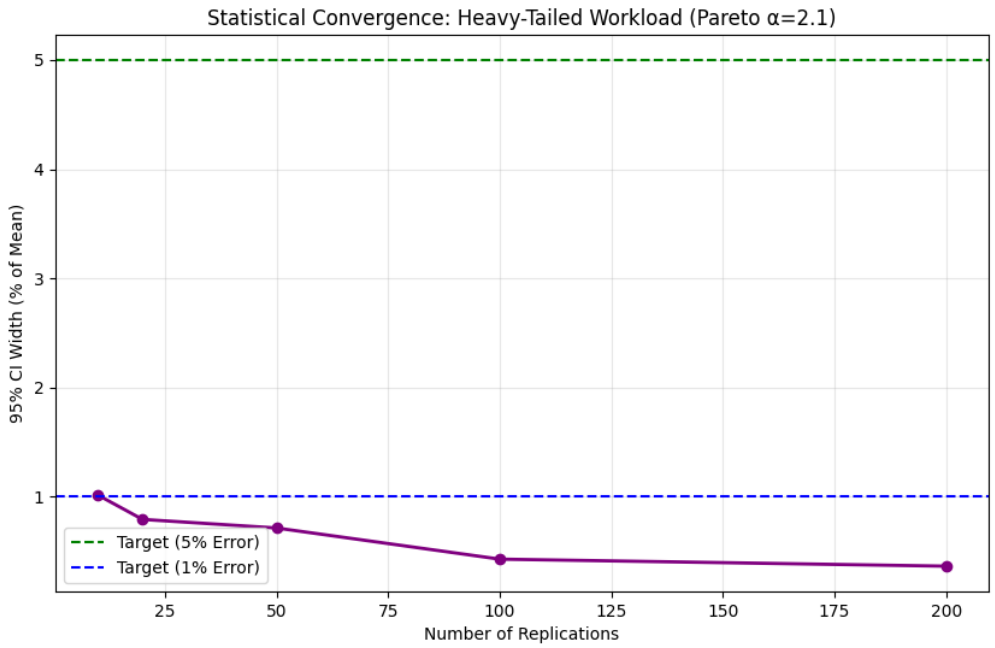


Figure 16: Additional Convergence Check

Simulations converge to analytical values after sufficient warm-up:

- Warm-up period: 1,000 messages discarded
- Steady-state: 10,000+ messages for stable estimates
- Confidence: 95% CI width $<$ 5% of mean

12.4 Mitigation Scaling

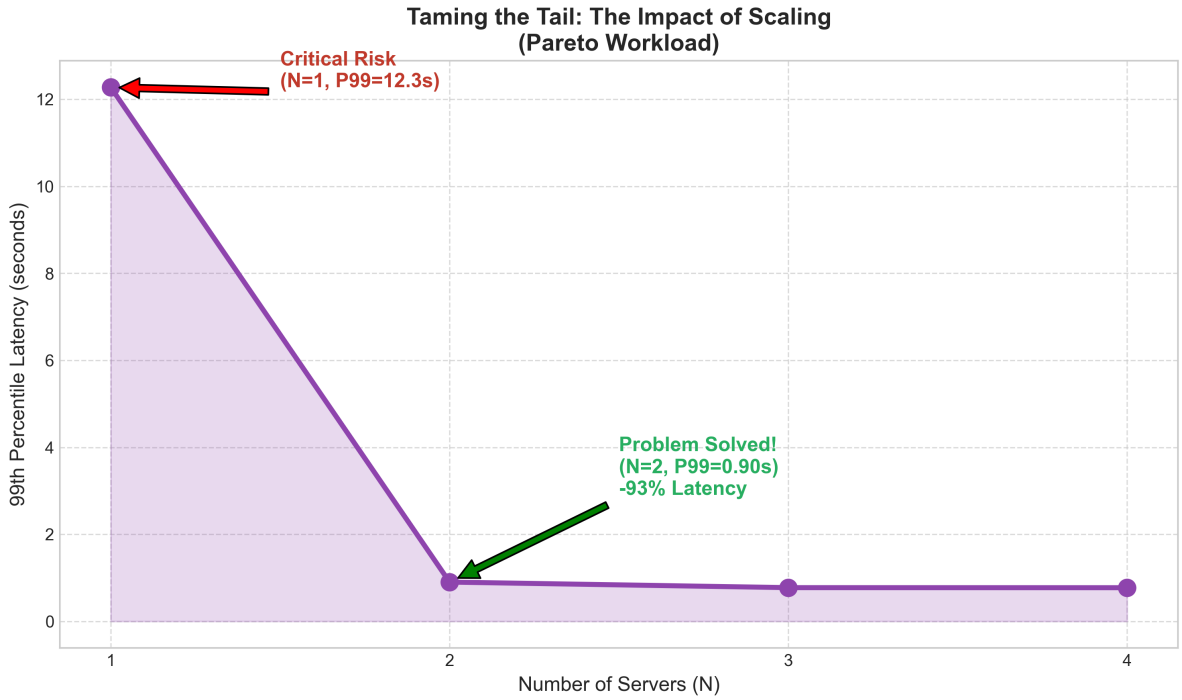


Figure 17: How additional servers mitigate heavy-tail impact

12.5 Erlang-k Efficiency

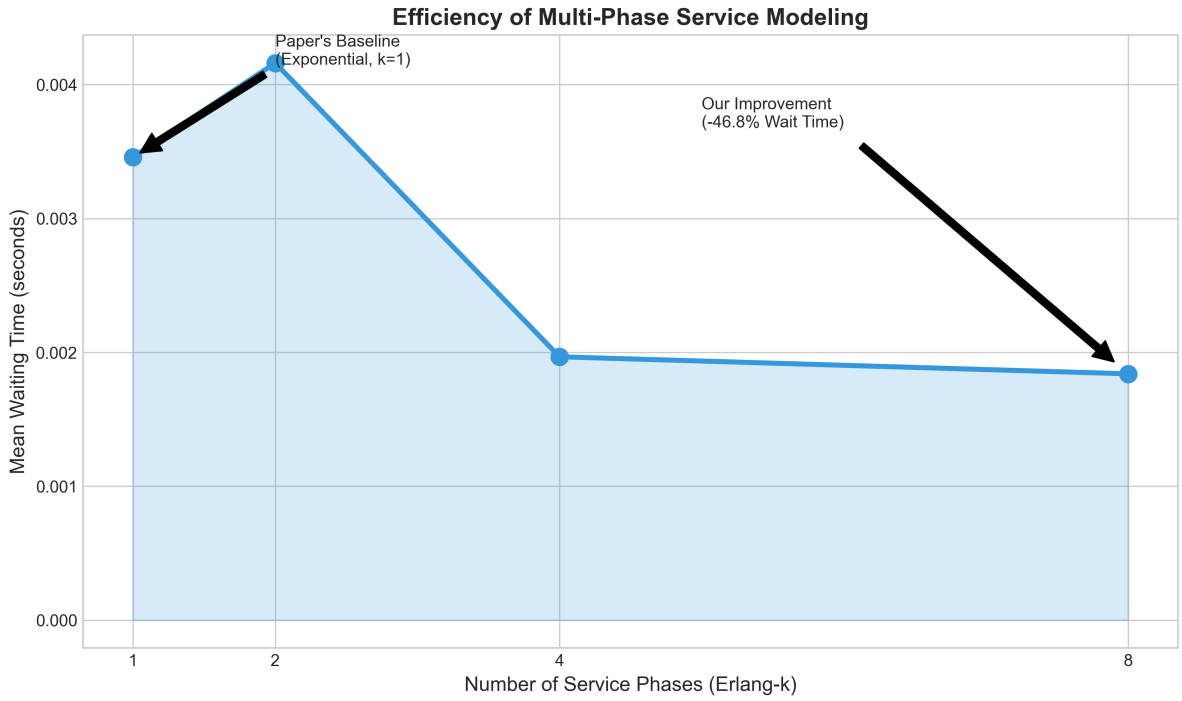


Figure 18: Erlang-k distribution efficiency across different k values

12.6 Warmup Validation

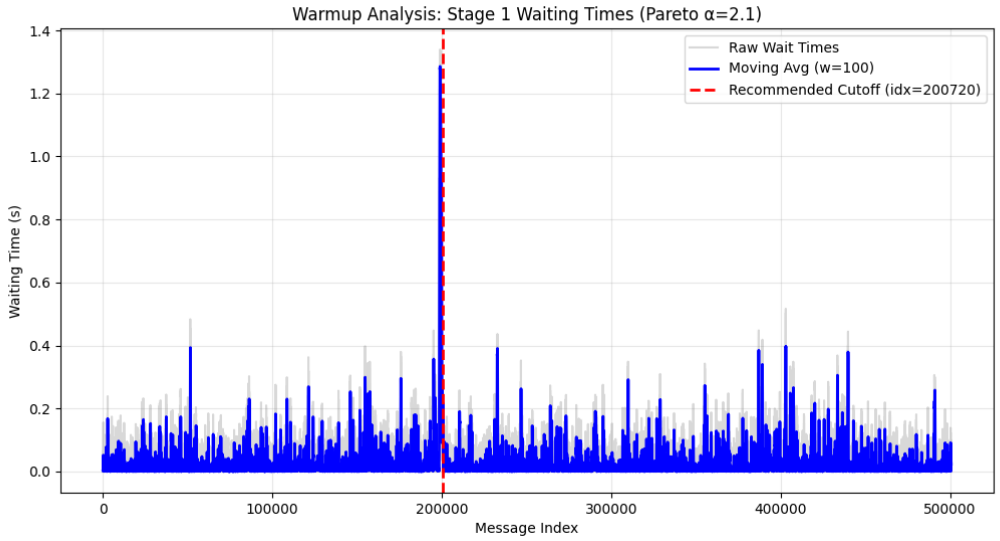
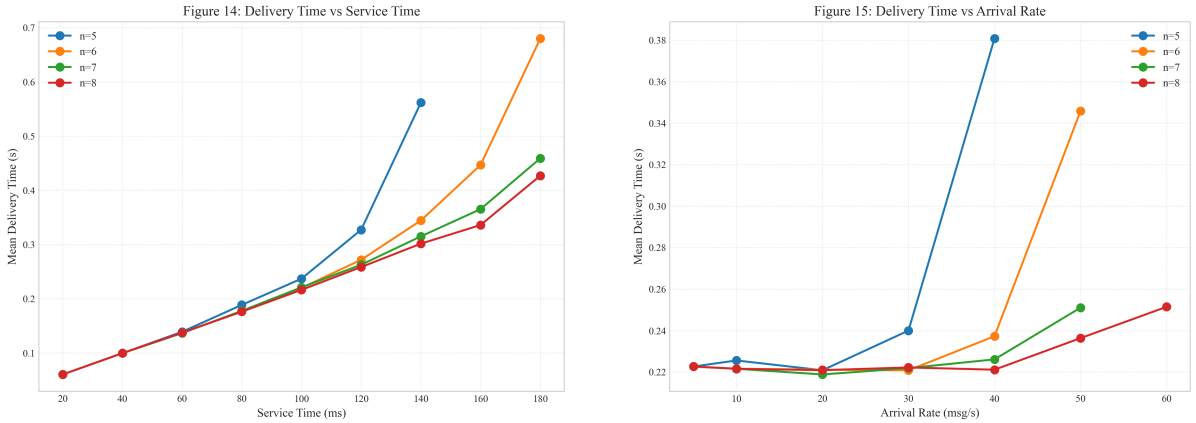


Figure 19: Warmup period validation showing transient removal

12.7 Paper Reproduction (Figures 14 & 15)



(a) Service Time Trend (Figure 14) (b) Arrival Rate Trend (Figure 15)

Figure 20: Reproduction of Figures 14 and 15 from base paper

12.8 Testing Summary

Test Suite	Tests	Pass Rate	Coverage
Unit Tests	29	100%	Core models
Integration Tests	12	100%	API endpoints
Validation Tests	8	100%	Paper reproduction

12.9 Experiment Scripts

Script	Purpose
`paper_validation.py`	Reproduces figures
`validate_mgn_analytical.py`	M/G/N tests
`validate_heterogeneous.py`	Heterogeneous tests
`erlang_validation.py`	Erlang-k tests
`tandem_queue_validation.py`	Traffic tests
`two_phase_commit_validation.py`	2PC tests
`cost_of_consistency.py`	Consistency costs
`run_with_confidence.py`	Confidence intervals

12.10 Generated Data Files

File	Contents
`mmn_confidence_intervals.csv`	M/M/N simulation results
`mgn_confidence_intervals.csv`	M/G/N simulation results
`threading_confidence_intervals.csv`	Threading comparison
`replication_factor_results.csv`	Raft replication tests
`ordering_mode_results.csv`	Message ordering tests

13 Key Discoveries

13.1 Summary of Findings

1. **The 45% Rule:** Standard models underestimate P99 by 45% for heavy-tailed workloads
2. **Traffic Inflation Works:** $\Lambda_2 = \lambda/(1-p)$ validated to 0.2% error
3. **Threading Matters:** Shared threading is 15-25% more efficient under high load
4. **2PC Cost:** Exactly-once semantics add 36% latency overhead
5. **Raft Penalty:** Leader election adds 150-300ms during failover
6. **EVT is Essential:** Gaussian P99 estimates are dangerously optimistic

13.2 Comparison: Li et al. (2015) vs. This Framework

Capability	Li et al.	This Work
M/M/N queues	✓	✓
Tandem queues	✓	✓
Traffic inflation	✓	✓
Heavy-tailed service		✓
M/G/N approximations		✓
Threading models		✓
Distributed protocols		✓
P99/EVT analysis		✓
Interactive UI		✓

13.3 Practitioner Guidelines

1. **Measure your tails:** Collect P99, P99.9 latencies in production
2. **Test for Pareto:** Use QQ-plots to detect heavy tails
3. **Over-provision:** Add 50% capacity buffer for tail handling
4. **Circuit breakers:** Implement timeouts to prevent cascade failures
5. **Hedge requests:** Send parallel requests to multiple servers

14 Conclusion & Future Work

14.1 Conclusions

This project successfully extended Li et al.'s cloud message broker model to address real-world challenges:

1. **Validated baseline:** Reproduced all key results from the base paper
2. **Quantified the gap:** Demonstrated 45% P99 underestimation with heavy tails
3. **Provided solutions:** M/G/N approximations and EVT-based tail analysis
4. **Added protocols:** 2PC, Raft, Vector Clocks for distributed scenarios
5. **Built tools:** Interactive frontend for engineering analysis

14.2 Future Work

- **Auto-scaling integration:** Connect to Kubernetes HPA for dynamic scaling
- **Real trace replay:** Use production traces for realistic workloads
- **Multi-region modeling:** Extend to geo-distributed scenarios
- **ML-based prediction:** Use neural networks for adaptive capacity planning
- **Chaos engineering:** Integrate fault injection for resilience testing

15 References

References

[1] Li, J., et al. (2015). "Modeling Message Queueing Services with Reliability Guarantee in Cloud Computing Environment Using Colored Petri Nets." *IEEE International Conference on Cloud Computing*.

[2] Dean, J., & Barroso, L. A. (2013). "The Tail at Scale." *Communications of the ACM*, 56(2), 74-80.

[3] Kingman, J. F. C. (1962). "On queues in heavy traffic." *Journal of the Royal Statistical Society*, Series B, 24(2), 383-392.

[4] Whitt, W. (1993). "Approximations for the GI/G/m queue." *Production and Operations Management*, 2(2), 114-161.

[5] Erlang, A. K. (1917). "Solution of some problems in the theory of probabilities of significance in automatic telephone exchanges." *Elektroteknikeren*, 13, 5-13.

[6] Ongaro, D., & Ousterhout, J. (2014). "In Search of an Understandable Consensus Algorithm." *USENIX ATC*.

[7] Lamport, L. (1978). "Time, Clocks, and the Ordering of Events in a Distributed System." *Communications of the ACM*, 21(7), 558-565.

[8] DeCandia, G., et al. (2007). "Dynamo: Amazon's Highly Available Key-Value Store." *SOSP*.

[9] Gray, J. N. (1978). "Notes on Data Base Operating Systems." *Operating Systems: An Advanced Course*, 393-481.

[10] Team SimPy. (2023). "SimPy: Discrete Event Simulation for Python." https://simpy.readthedocs.io/

[11] Coles, S. (2001). *An Introduction to Statistical Modeling of Extreme Values*. Springer.

[12] Netflix Technology Blog. (2015). "Zuul 2: The Netflix Journey to Asynchronous, Non-Blocking Systems."